

# Debugging Broken Pipes

App & System Services

Core OS

Kernel



DTS Engineer

Apple

Created Replies Boosts Views Participants

Jan '25 0 0 427 1

SIGPIPE is an ongoing source of grief on Apple systems [1]. I've talked about it numerous times here on the forums. It cropped up again today, so I decided to collect my experiences into one post.

If you have questions or comments, please put them in a new thread. Put it in the [App & System Services > Core OS](#) topic area so that I see it.

Share and Enjoy

Quinn "The Eskimo!" @ Developer Technical Support @ Apple  
let myEmail = "eskimo" + "1" + "@" + "apple.com"

[1] Well, on Unix-y systems in general, but my focus is Apple systems (-:

---

## Debugging Broken Pipes

On Unix-y systems, writing to a pipe whose read side is closed will raise a SIGPIPE signal. The default disposition of that signal is to terminate your process [1]. *Broken pipe* terminations are tricky to debug on Apple systems because the termination doesn't generate a crash report.

For example, consider this code:

```
1 let (read, write) = try FileDescriptor.pipe()
2
3 // This write works.
4
5 try write.writeAll("Hello Cruel World!".utf8)
6 let msg = try read.read(maxCount: 256)
7 ... do something with `msg` ...
8
9 // But if you close the read side...
10
11 try read.close()
12
13 // ... the write call raises a `SIGPIPE`.
14
15 try write.writeAll("Goodbye Cruel World!".utf8)
```

**Note** This code relies on some extensions to `FileDescriptor` type that make it easier to call the `pipe` and `write` system calls. For more information about how I set that up, see [Calling BSD Sockets from Swift](#).

If you put this in an iOS app and run it outside of Xcode, the app will terminate without generating a crash report.

This logic also applies to BSD Sockets. Writing to a disconnected socket may also trigger a SIGPIPE. This applies to the `write` system call and all the `send` variants: `send`, `sendto`, and `sendmsg`.

**IMPORTANT** Broken pipe terminations are even more troubling with sockets because sockets are commonly used for networking, where you have no control over the remote peer.

It's easy to reproduce this signal with Unix domain sockets:

```

1  let (read, write) = try FileDescriptor.socketPair(AF_UNIX, SOCK_STREAM, 0)
2
3  // This write works.
4
5  try write.writeAll("Hello Cruel World!".utf8)
6  let msg = try read.read(maxCount: 256)
7  ... do something with `msg` ...
8
9  // But if you close the read side...
10
11 try read.close()
12
13 // ... the write call raises a `SIGPIPE`.
14
15 try write.writeAll("Goodbye Cruel World!".utf8)

```

However, this isn't limited to just Unix domain sockets; TCP sockets are a common source of broken pipe terminations.

[1] At first blush this API design might seem bananas, but it kinda makes sense in the context of traditional Unix command-line tools.

#### Confirm the Problem

The primary symptom of a broken pipe problem is that your app terminates without generating a crash report. Unfortunately, that's not definitive. There are other circumstances where your app can terminate without generating a crash report. For example, another common cause of such terminations is the app calling `exit`.

There are two ways you can confirm this problem. The first relies on Xcode. Run your app in the Xcode debugger and, if it suddenly stops with the message `Terminated due to signal 13`, you know you've been terminated because of a broken pipe.

**IMPORTANT** Double check that the signal number is 13, the value of `SIGPIPE`.

If you can't reproduce the problem in Xcode, look in the system log. When an app terminates the system records information about the reason. The exact log message varies from platform to platform, and from OS version to OS version. However, in the case of a `SIGPIPE` termination there's usually a log entry containing `PIPE` or `SIGPIPE`, or that references signal 13.

For example, on iOS 18.2.1, I see this log entry:

```

1  type: default
2  time: 11:59:00.321882+0000
3  process: SpringBoard
4  subsystem: com.apple.runningboard
5  category: process
6  message: Firing exit handlers for 16876 with context <RBSPProcessExitContext| specific, status:<RBSPProcessExitStatus| don

```

The log message contains both `SIGPIPE` and the `SIGPIPE` signal number, 13.

For more information about accessing the system log, see [Your Friend the System Log](#).

#### Locate the Problem

Once you've confirmed that you have a broken pipe problem, you need to locate the source of it. That is, what code within your process is writing to a broken pipe?

If you can reproduce the problem in Xcode, configure LLDB to stop on `SIGPIPE` signals:

```

1  (lldb) process handle -s true SIGPIPE
2  NAME      PASS  STOP  NOTIFY
3  =====  =====
4  SIGPIPE   true  true  false

```

When the process writes to a broken pipe, Xcode stops in the debugger. Look at the backtrace in the Debug navigator to find the offending write.

If you can't reproduce the problem in Xcode, one option is to add a signal handler that catches the `SIGPIPE` and triggers a crash. For example:

```

1  #include <signal.h>
2
3  static void sigpipeHandler(int sigNum) {
4      __builtin_trap();
5  }
6
7  extern void installSIGPIPEHandler(void) {
8      signal(SIGPIPE, sigpipeHandler);
9  }

```

Here the signal handler, `sigpipeHandler`, forces a crash by calling the `__builtin_trap` function.

**IMPORTANT** This code is in C, and uses `__builtin_trap` rather than `abort`, because of the very restricted environment in which the signal handler runs [1].

With this signal handler in place, writing to a broken pipe generates a crash report. Within that crash report, the crashing thread backtrace gives you a hint as to the location of the offending write. For example:

```

1  0 SIG-PIPETest      ... sigpipeHandler + 8
2  1 libsystem_platform.dylib ... _sigtramp + 56
3  2 libswiftSystem.dylib ... closure #1 in FileDescriptor._writeAll<A>(_:) + 100
4  3 libswiftSystem.dylib ... partial apply for closure #1 in FileDescriptor._writeAll<A>(_:) + 20
5  4 libswiftSystem.dylib ... partial apply for closure #1 in Sequence._withRawBufferPointer<A>(_:) + 108
6  5 libswiftCore.dylib ... String.UTF8View.withContiguousStorageIfAvailable<A>(_:) + 108
7  6 libswiftCore.dylib ... protocol witness for Sequence.withContiguousStorageIfAvailable<A>(_:) in conform...
8  7 libswiftCore.dylib ... dispatch thunk of Sequence.withContiguousStorageIfAvailable<A>(_:) + 32
9  8 libswiftSystem.dylib ... Sequence._withRawBufferPointer<A>(_:) + 472
10 9 libswiftSystem.dylib ... FileDescriptor._writeAll<A>(_:) + 104
11 10 SIG-PIPETest      ... FileDescriptor.writeAll<A>(_:) + 28
12 ...

```

**Note** The `write` system call is not shown in the backtrace. That's because the crash reporter is not backtracing correctly across the signal handler stack frame that was inserted by the kernel between frames 1 and 2 [1]. Fortunately that doesn't matter here, because we primarily care about *our* code, which is visible in frame 10.

I can't see any problem with putting this code in your development build, or even deploying it to your beta testers. Think carefully before putting it in a production build that you deploy to all your users. Signal handlers are tricky [1].

[1] For all the gory details on that topic, see [Implementing Your Own Crash Reporter](#) for more information about that issue.

[2] This is one of the gory details covered by [Implementing Your Own Crash Reporter](#).

#### Resolve the Problem

The best way to resolve this problem depends on whether it's being caused by a pipe or a socket. The socket case is easy: Use the `SO_NOSIGPIPE` socket option to disable `SIGPIPE` on the socket. Once you do that, writing to the socket when it's disconnected will return an `EPIPE` error rather than raising the `SIGPIPE` signal.

For example, you might tweak the code above like so:

```

1  let (read, write) = try FileDescriptor.socketPair(AF_UNIX, SOCK_STREAM, 0)
2  try read.setSocketOption(SOL_SOCKET, SO_NOSIGPIPE, 1 as CInt)
3  try write.setSocketOption(SOL_SOCKET, SO_NOSIGPIPE, 1 as CInt)

```

**Note** Again, this is using helpers from [Calling BSD Sockets from Swift](#).

The situation with pipes is tricky. Apple systems have no way to disable `SIGPIPE` on a pipe, leaving you with two less-than-ideal options:

- Disable `SIGPIPE` globally. To do this, call `signal` with `SIG_IGN`:

```
signal(SIGPIPE, SIG_IGN)
```

The downside to this approach is that affects the entire process. You can't, for example, use this technique in library code.

- Switch to Unix domain sockets. Rather than use a pipe for your IPC, use Unix domain sockets instead. As they're both file descriptors, it's usually quite straightforward to make this change.

The downside here is obvious: You need to modify your IPC code. That might be problematic, for example, if this IPC code is embedded in a framework that you don't build from source.